PROJECT [OS] :: Managed Memory with More Concurrency - and Threads

Introduction

In this project, you will continue from your previous managed memory and multiprocessing assignment. That earlier project exposed the challenges of maintaining allocator consistency when multiple processes operate on separate memory spaces. In this follow-up, you will receive a complete working solution as your starting point. The provided version uses a shared heap and synchronization to coordinate concurrent access across processes. Your task is to study this correct but inefficient implementation, then extend and refine it: first by converting it to a threaded version using pthreads, and later by reducing lock contention to improve concurrency and performance.

Memory Management and fork()

In the previous project, your allocator functioned correctly for small tests but failed under heavier multiprocessing workloads. The root cause lies in how `fork()` manages memory. When a process calls `fork()`, the operating system creates a new process whose address space is a copy of the parent's, implemented through copy-on-write. Initially, parent and child share the same physical pages, but once either modifies memory, those pages are duplicated privately. This means that while each process starts with an identical view of the heap, any updates to allocator metadata—such as the free list or block headers—are isolated to that process. Each child believes it is managing the same heap, but in reality, they are modifying separate copies, leading to allocator corruption and crashes.

A potential fix would be to reinitialize the allocator after each `fork()`, so every process manages its own private heap. However, that does not reflect the model of concurrency we intend to study. The goal here is to implement a true shared-memory system in which multiple processes operate cooperatively on a single heap. This is achieved by placing the allocator's data structures inside a shared `mmap()` region, allowing all processes to see and modify the same memory. Once memory is truly shared, synchronization becomes essential: without coordination, simultaneous access would destroy allocator state. The simplest solution is coarse-grained locking, where a single global lock protects all allocator operations. Later, you will extend this design toward finer-grained synchronization and compare the shared-memory process model with the threaded model, where threads naturally share memory within one address space.

Project Setup and Getting Started

This project begins with a complete, working solution that demonstrates how to use a shared heap allocator across multiple processes. The provided code already implements shared memory using `mmap()` and synchronizes allocator operations with a process-shared semaphore. Your role is to study this implementation, understand its structure, and then extend it to explore concurrency and performance under different models. While the program runs correctly, it is intentionally inefficient—this inefficiency is the foundation for the work you will perform in later parts of the project.

1. Downloading and Inspecting the Starting Code

Download the provided scaffold file **sharedhash.c** from Canvas. This is a fully functional version of the shared-memory Huffman hashing program. You must work directly within this file. Do not rename it, split it into multiple files, or alter its structure beyond the clearly marked student sections. The autograder depends on the file name and format, and changing either may result in a failing grade.

Once downloaded, read through the file carefully before compiling. The file is heavily commented to guide you through the allocator, synchronization primitives, and execution flow. You are expected to understand the control flow from `main()` through `run_multi()` and `process_block()` before modifying anything.

```
gcc -Wall -Wextra -o sharedhash sharedhash.c
./sharedhash sample.txt
./sharedhash sample.txt -m
```

The `-m` flag activates multi-process mode, launching a new process for each 1KB block in the input file. If you compile and run with `-DDEBUG=2`, the program prints process IDs, letting you confirm that multiple processes are running concurrently.

```
gcc -DDEBUG=2 -Wall -Wextra -o sharedhash sharedhash.c
./sharedhash sample.txt -m
```

Expected output should include lines like:

```
[PID 10435] Block 0 hash: 338324
[PID 10436] Block 1 hash: 881232
[PID 10437] Block 2 hash: 765411
Final signature: 1984967
```

Each child process computes the hash for one block independently, then writes its result back to the parent through a pipe. The parent accumulates these values and prints the final signature. The important property is that this signature must be identical in both single-process and multi-process modes.

2. Understanding the Shared Memory Allocator

The allocator is built on `mmap()` to create a shared memory region visible to all processes. Unlike `malloc()`, which allocates memory from a process's private heap, `mmap()` with the `MAP_SHARED` and `MAP_ANONYMOUS` flags allows multiple processes to access the same physical memory.

Internally, the allocator maintains a single linked list of free blocks—an *embedded free list*. Each free block stores its size and a pointer to the next free block, all within the managed memory region itself. This design avoids using any external dynamic allocations. The same region also holds the data allocated for the Huffman tree, heap nodes, and other structures.

This is a **first-fit** allocator: it scans the free list from the beginning and takes the first block large enough to satisfy the request. If the block is larger than required, it splits it into an allocated region and a smaller free block. When a block is freed, the allocator inserts it back into the free list in address order and attempts to coalesce adjacent blocks.

Because the heap is shared across processes, every operation on the free list is protected by a semaphore. The semaphore itself is stored in shared memory, so that all processes synchronize on the same lock. This ensures correctness but introduces significant performance overhead—only one process at a time can allocate or free memory.

```
// Simplified pattern (already implemented)
void *umalloc(size_t size) {
    sem_wait(mLock);     // Acquire global lock
    void *ptr = _umalloc(size);  // Perform allocation
    sem_post(mLock);     // Release lock
    return ptr;
}

void ufree(void *ptr) {
    sem_wait(mLock);
    _ufree(ptr);
    sem_post(mLock);
}
```

Your later tasks will focus on reducing this lock contention, but for now, you should confirm that this mechanism prevents memory corruption and maintains correctness.

3. Building Confidence with Incremental Testing

This project is complex. Treat it as a systems lab, not a single coding exercise. You will be working with `mmap()`, process control, and synchronization, all of which can fail in ways that are hard to diagnose. Build confidence gradually. Follow this general workflow:

1. Verify that the provided code compiles without warnings and runs correctly in both single and multi modes.
2. Use `valgrind` to check for memory errors and leaks in single mode.
3. Re-run with `-m` and verify that the final signature matches the single-process version.

4. Use `ps` or `top` to confirm multiple processes are active concurrently.
5. Experiment with different input sizes (e.g., 1KB, 100KB, 1MB) to observe performance changes.

For instance:

```
# Generate test files
dd if=/dev/urandom of=test_1KB.bin bs=1024 count=1
dd if=/dev/urandom of=test_100KB.bin bs=1024 count=100
dd if=/dev/urandom of=test_1MB.bin bs=1024 count=1024

# Test correctness
./sharedhash test_1KB.bin
./sharedhash test_1KB.bin -m

# Test with debugging
gcc -DDEBUG=2 -o sharedhash sharedhash.c
./sharedhash test_100KB.bin -m
```

If both runs produce the same final signature, your allocator and synchronization mechanisms are functioning correctly. If they differ, the most likely cause is a race condition or corruption in the free list.

4. Debugging Memory Errors

Memory bugs are common in this type of project and can be subtle. Use **Valgrind** to detect invalid reads, writes, and memory leaks. Remember that Valgrind will not always interpret custom allocators correctly—it can report false positives when you use `mmap()`-based memory management—but it remains invaluable for finding invalid pointer usage.

```
valgrind --leak-check=full --show-leak-kinds=all ./sharedhash test_1KB.bin
```

If Valgrind reports “Invalid read of size 8” or “Invalid write of size 8,” the most common causes are:

- Overwriting allocator metadata (e.g., writing into a freed block)
- Using unaligned pointers when splitting blocks
- Corrupted `next` pointers in the free list
- Failing to acquire the semaphore before calling `umalloc()` or `ufree()`

To help you locate these errors, use print statements carefully. For instance, you can temporarily instrument `_umalloc()` and `_ufree()` with diagnostic output:

```
fprintf(stderr, "Allocating %zu bytes at %p (free list head: %p)\n", size, ptr, *free_list_ptr);
```

Always remove debugging output before submission. The autograder expects output to match the reference format exactly.

5. Asking Good Questions

Asking for help effectively is part of this project's learning goal. When you get stuck, focus on presenting evidence rather than symptoms. Instead of saying "my code crashes," describe what you tested, what you expected, and what actually happened. For example:

> "I ran `./sharedhash test_100KB.bin -m` and confirmed that both the single and multi versions produce different signatures. I added a print statement inside `_ufree()` and noticed that after freeing the third block, the `next` pointer points to `0xDEADBEEF`. This suggests that my coalescing logic is touching allocated memory."

That kind of description makes it much easier for an instructor or TA to guide you toward the root cause.

6. Experimentation and Performance Observation

The current implementation uses a single global semaphore to protect the allocator. This guarantees correctness but eliminates most of the performance benefit of running in parallel. You can measure this effect directly using `time`:

```
time ./sharedhash test_1MB.bin
time ./sharedhash test_1MB.bin -m
```

You may notice that the multi-process version is not significantly faster—and may even be slower—than the single-process version. That's expected. The cost of locking, context switching, and process creation dominates the small parallel gains. This inefficiency is intentional: in the next phase, you will explore strategies to improve concurrency by reducing contention and comparing process-based and thread-based models.

For deeper analysis, you can instrument the allocator to count how often the semaphore is acquired. For example:

```
#include <stdatomic.h>

atomic_int lock_acquisitions = 0;

void *umalloc(size_t size) {
    atomic_fetch_add(&lock_acquisitions, 1);
    sem_wait(mLock);
    void *p = _umalloc(size);
    sem_post(mLock);
    return p;
}
```

At the end of `main()`, print this value to see how many times the allocator lock was acquired. For larger files, you may see tens of thousands of lock acquisitions, showing that nearly every memory operation is serialized.

7. Compilation and Environment Notes

All grading and testing will be performed on Linux using `gcc`. You may develop on macOS, but note that:

- `valgrind` is unreliable on macOS and may fail entirely.
- macOS uses a different implementation of `mmap()` that sometimes behaves differently with shared memory.
- Semaphore semantics are not identical across platforms; `sem_init()` with `pshared=1` is not supported on all macOS versions.

For these reasons, testing your final implementation in the Linux lab (or on a virtual machine running Linux) is mandatory. Platform-specific differences will not be accepted as valid reasons for failing tests.

8. Project Expectations

This project is designed to develop your intuition about shared memory, synchronization, and allocator design. While the codebase looks intimidating, much of it is provided scaffolding. Your focus should be on understanding what each component does and how data flows through the system. Specifically:

- Be able to explain how the allocator manages free and allocated blocks.
- Understand how the semaphore prevents corruption when multiple processes call `umalloc()` or `ufree()`.
- Recognize why buffer allocation with `umalloc()` is necessary before `fork()`.
- Appreciate that correctness comes before performance—your modifications must always produce the same final signature as the baseline version.

Before you begin making changes, ensure that your copy of the baseline program runs correctly and passes all correctness checks. Only once you have verified that behavior should you move on to experimenting with modifications. As the complexity of your implementation increases, rigorous testing will become more important. Save known-good versions frequently and use `git` or versioned backups to avoid losing working code.

Finally, understand that this assignment reflects real systems work. Allocators, synchronization primitives, and concurrent data structures are cornerstones of operating systems and runtime libraries. Debugging them is challenging and time-consuming—but this difficulty is what makes them valuable learning experiences.

Part 1: Converting the Shared Memory Hasher to Threads

In the provided **sharedhash.c** program, each 1KB block of input is processed by a separate `fork()`ed process. These child processes share access to a global heap allocator through `mmap()`-based shared memory, coordinating access with a process-shared semaphore. This model is correct but costly. Every `fork()` call duplicates address space structures, incurs kernel overhead, and results in heavyweight synchronization. In this part, you will convert the program from a multi-process architecture to a multi-threaded one using the `pthread` library. The goal is to preserve correctness while eliminating the unnecessary complexity and overhead of inter-process communication.

1. Conceptual Shift: Processes vs. Threads

Before making changes, it's essential to understand how threads differ from processes. Processes have independent address spaces; after `fork()`, memory is only shared when explicitly mapped. Threads, on the other hand, automatically share their entire address space, including the heap, globals, and static variables. This means:

- You will **remove all `mmap()` calls** that were used solely for sharing memory between processes.
- The allocator will naturally operate on shared memory—no special setup is needed for inter-thread visibility.
- The semaphore used for process coordination will be replaced by a **pthread mutex**.

The practical effect is a much simpler setup: instead of creating shared memory manually, you will rely on the implicit shared address space that all threads inhabit.

```
// Old (process-shared)
sem_t* mLock;
sem_init(mLock, 1, 1);  // pshared = 1

// New (thread-shared)
pthread_mutex_t mLock = PTHREAD_MUTEX_INITIALIZER;
```

2. Step-by-Step Conversion Plan

Follow this sequence carefully. Each step has been designed to minimize confusion and prevent cascading bugs.

Step 1: Replace Process Creation with Threads

In the original code, `run_multi()` uses `fork()` to spawn a new process for each block and pipes for communication. In your thread-based version, you will replace this with `pthread_create()` calls. Each thread will handle one block of data and write its hash result into a shared array.

```
// Thread argument structure
typedef struct {
    int block_id;
    unsigned char *block_buf;
    size_t block_len;
    unsigned long *results;
} thread_arg_t;

// Worker thread function
void *worker_thread(void *arg) {
    thread_arg_t *targ = (thread_arg_t *)arg;
    unsigned long h = process_block(targ->block_buf, targ->block_len);
    ufree(targ->block_buf);
    targ->results[targ->block_id] = h;
    return NULL;
}
```

Each thread computes its hash independently. The parent thread (the main program) will later join all threads and combine their results into a final hash signature.

Step 2: Remove Pipes and Replace with Shared Results

In the multi-process version, pipes were used to collect results from children. With threads, you can replace all pipe logic with a simple array of `unsigned long` results.

```
// Global or stack array
unsigned long results[1024];

// After joining threads
unsigned long final_hash = 0;
for (int i = 0; i < num_blocks; i++) {
    print_intermediate(i, results[i], 0);
    final_hash = (final_hash + results[i]) % LARGE_PRIME;
}
print_final(final_hash);
```

Since all threads share memory, no explicit inter-thread communication mechanism (like pipes) is needed. Each thread writes its result directly to its own index in the array.

Step 3: Replace Semaphore with a Mutex

Your allocator currently uses `sem_wait()` and `sem_post()` to protect critical sections. Replace them with `pthread_mutex_lock()` and `pthread_mutex_unlock()`. Also, remove `sem_init()` from the setup code and replace it with a static mutex declaration.

```
// Replace:
sem_wait(mLock);
...
sem_post(mLock);

// With:
pthread_mutex_lock(&mLock);
...
pthread_mutex_unlock(&mLock);
```

For single-threaded mode, you can keep the same `use_multiprocess` flag (rename it to `use_multithread` if desired). Conditional locking ensures the single-threaded path avoids unnecessary synchronization overhead.

Step 4: Replace Shared Memory Setup with Local Allocations

In `init_umem()`, remove all `mmap()` calls and replace them with regular `malloc()`. Threads naturally share global variables, so there's no need for shared memory mappings.

```
static node_t *free_list = NULL;

void *init_umem(void) {
    void *base = malloc(UMEM_SIZE);
    if (!base) {
        perror("malloc");
        exit(1);
    }
    free_list = (node_t *)base;
    free_list->size = UMEM_SIZE - sizeof(node_t);
    free_list->next = NULL;
    return base;
}
```

By converting `free_list_ptr` (a pointer-to-pointer) to a regular pointer, you eliminate unnecessary indirection. Threaded programs inherently share the same memory space, so a global pointer suffices.

Step 5: Convert `run_multi()` into `run_threads()`

The main driver for the multi-threaded version should closely mirror the structure of the process-based version. You'll still read the file block-by-block, allocate memory for each, and dispatch workers. The key differences are that you now:

- Call `pthread_create()` instead of `fork()`
- Pass thread arguments instead of writing to pipes
- Use `pthread_join()` instead of `waitpid()`

Step 6: Update `main()`

Your `main()` function should select between single-threaded and multi-threaded modes based on the command-line flag `-t` (for "threaded"). For compatibility, you can keep the existing `-m` for processes, although the autograder will focus on the threaded mode.

```
int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s  [-m|-t]\n", argv[0]);
        return 1;
    }

    const char *filename = argv[1];
    use_multiprocess = (argc >= 3 && strcmp(argv[2], "-m") == 0);
    int use_multithread = (argc >= 3 && strcmp(argv[2], "-t") == 0);
```

```
    init_umem();

    if (use_multithread)
        return run_threads(filename);
    else if (use_multiprocess)
        return run_multi(filename);
    else
        return run_single(filename);
}
```

3. Testing Your Threaded Implementation

Once your thread-based version compiles, begin testing gradually:

1. Run in single-threaded mode and verify it matches the baseline output.
2. Run with `-t` using small files (1–10KB).
3. Increase input size to confirm stability under higher concurrency.
4. Use `valgrind` to detect memory leaks or invalid access patterns.

If your threaded version produces different hashes than the process version, possible causes include:

- Race conditions due to missing or incorrect locks in the allocator
- Incorrect indexing of the `results` array
- Reusing the same buffer pointer for multiple threads
- Not joining all threads before aggregating results

4. Expected Behavior and Performance

The threaded version should behave identically to the process-based one from a correctness perspective: the same input file should yield the same final hash signature. However, you should notice a difference in performance and resource usage:

- **Lower overhead:** Threads are lighter than processes and share code, data, and file descriptors.
- **No pipes or copies:** Data sharing is direct, not via interprocess communication.
- **Improved responsiveness:** Fewer system calls and context switches.
- **Limited speedup:** Still limited by the coarse-grained allocator lock.

Run a few timing tests to confirm the expected behavior:

```
time ./sharedhash test_1MB.bin
time ./sharedhash test_1MB.bin -m
time ./sharedhash test_1MB.bin -t
```

You may observe that the threaded version is slightly faster than the process version but still far from ideal scaling. That’s expected—the single global mutex in the allocator still forces threads to wait for one another. In the next part, you’ll redesign the allocator to reduce this contention and measure actual speedup.

5. Common Mistakes and Debugging Advice

The following issues are common in this conversion:

- **Stack variable lifetime:** Do not pass the address of a stack variable (like `thread_arg_t arg;`) to `pthread_create()`. Use an array of argument structs so that each thread gets a stable address.
- **Forgetting to join threads:** Threads that aren't joined can cause undefined behavior or missing results.
- **Mixing semaphores and mutexes:** After conversion, ensure there are no lingering `sem_*` calls.
- **Not freeing memory:** Verify that `ufree()` is called for each `umalloc()` allocation to avoid leaks.
- **Uninitialized globals:** Remember that the mutex must be initialized statically or with `pthread_mutex_init()`.

6. Deliverables for Part 1

Your submission for Part 1 should include:

- A working `sharedhash.c` that supports `-t` for threaded execution.
- Output from correctness tests confirming that single, process, and thread modes produce identical final signatures.
- A brief explanation (1–2 paragraphs) describing the main changes you made and how thread-based memory sharing differs from process-based shared memory.
- All writeup must be submitted in a markdown file (.md)

7. Grading Criteria

Points for Part 1 are awarded based on:

- Correctness: identical outputs across all modes
- Proper use of pthread APIs and mutexes
- Memory safety
- Code readability and comments
- Successful test results and explanation

Passing this stage confirms that you understand the fundamental differences between process and thread concurrency and can implement a correct, shared-memory allocator in both environments. The next part focuses on improving performance by reducing lock contention and introducing finer-grained synchronization.

Part 2: Exploring Concurrency and Optimizing the Allocator

In Part 1, you converted the shared memory hasher from a process-based program to a thread-based one. Your solution was correct and functional, but its performance remained poor. Nearly every call to `umalloc()` and `ufree()` required a global mutex lock, forcing all threads to take turns accessing the allocator. This means that even though your threads ran concurrently, they could not make real progress in parallel. In this final part, you will investigate ways to reduce this bottleneck by experimenting with allocator-level optimizations. The goal is to explore trade-offs in concurrency—not necessarily to build the fastest allocator, but to reason about why it performs the way it does.

1. Starting Point and File Naming

Begin with your completed **sharedhash.c** from Part 1. Create a copy named **esharedhash.c**. This will be your experimental version. You will submit both files:

- `sharedhash.c` — your correct, fully working threaded implementation from Part 1
- `esharedhash.c` — your modified experimental version for this part

This separation allows you to try aggressive changes without breaking your working baseline. You should be able to recompile and compare both versions side by side:

```
gcc -pthread -o sharedhash sharedhash.c
gcc -pthread -o esharedhash esharedhash.c
```

Then test them with:

```
time ./sharedhash test_1MB.bin -t
time ./esharedhash test_1MB.bin -t
```

Your goal is not just to make `esharedhash` faster, but to understand *why* it behaves differently.

2. The Bottleneck: Coarse-Grained Locking

In the baseline implementation, the allocator uses one global lock (`pthread_mutex_t mLock`) to protect the entire free list. This ensures safety but serializes every allocation and free operation. With 10 threads performing hundreds of allocations each, the lock becomes a choke point. When one thread calls `umalloc()`, all other threads block, even if they are allocating memory from unrelated parts of the heap.

You can confirm this behavior by adding an atomic counter around the lock calls:

```
#include <stdatomic.h>

atomic_int lock_count = 0;

void *umalloc(size_t size) {
    atomic_fetch_add(&lock_count, 1);
    pthread_mutex_lock(&mLock);
```

```
    void *ptr = _umalloc(size);
    pthread_mutex_unlock(&mLock);
    return ptr;
}
```

At the end of `main()`, print the number of lock acquisitions and compare the counts between `sharedhash` and `esharedhash`. If your changes work, `esharedhash` should show significantly fewer lock acquisitions per run.

3. Objective of Part 2

Your task is to redesign or modify the allocator to reduce contention while preserving correctness. There are multiple directions you can explore. You are free to choose your approach, but you must explain and justify it in your write-up. The only hard requirements are:

- Your allocator must remain memory safe—no corruption, no invalid reads or writes.
- Your program must still produce the same final hash signature as before.
- Your code must compile cleanly with `-pthread` and `-Wall` flags.
- You must demonstrate a measurable difference in performance and explain it.

The following sections describe several strategies you can try. Choose one and explore it in depth, or experiment with multiple if you have time.

4. Strategy A: Per-Thread Memory Pools

The simplest and most effective optimization is to eliminate the global lock by giving each thread its own allocator region. Since threads do not interfere with one another, most `umalloc()` and `ufree()` calls can proceed without synchronization.

Concept

Divide the shared heap into *N* equal regions, one per thread. Each thread maintains its own private free list or bump pointer within its region. Allocations and frees within that region require no locking. Only when a thread's region runs out of space should it acquire the global lock and request additional memory.

Implementation Sketch

```
__thread char *pool_start = NULL;
__thread char *pool_current = NULL;
__thread size_t pool_size = 0;

void thread_pool_init(int tid, int num_threads) {
    size_t chunk = UMEM_SIZE / num_threads;
    pool_start = (char *)global_heap + (tid * chunk);
    pool_current = pool_start;
    pool_size = chunk;
}

void *umalloc_fast(size_t size) {
```

```
    size = ALIGN(size);
    if (pool_current + size < pool_start + pool_size) {
        void *ptr = pool_current;
        pool_current += size;
        return ptr;
    }
    pthread_mutex_lock(&mLock);
    void *ptr = _umalloc(size);
    pthread_mutex_unlock(&mLock);
    return ptr;
}
```

This design removes the lock for most allocations, keeping the global lock as a fallback. You can ignore `ufree()` or make it a no-op for this experiment (acceptable, since the program exits soon after computation). Be sure to initialize the per-thread pool inside your thread's start routine.

Trade-offs

- Very fast for workloads with independent allocations.
- Wastes memory if threads allocate unevenly.
- May require tuning pool sizes for different file sizes.

5. Strategy B: Fine-Grained Locking

Another approach is to keep a single shared heap but reduce contention by locking smaller regions independently. Instead of one lock for the entire free list, maintain multiple locks—one per size class or heap segment. Threads working with small allocations won't block those working with large ones.

```
#define NUM_CLASSES 8
pthread_mutex_t locks[NUM_CLASSES];
node_t *free_lists[NUM_CLASSES];

int size_class(size_t size) {
    if (size <= 32) return 0;
    if (size <= 64) return 1;
    if (size <= 128) return 2;
    ...
    return NUM_CLASSES - 1;
}

void *umalloc(size_t size) {
    int c = size_class(size);
    pthread_mutex_lock(&locks[c]);
    void *ptr = allocate_from_list(free_lists[c], size);
    pthread_mutex_unlock(&locks[c]);
    return ptr;
}
```

This design improves concurrency but increases complexity. Each free list must be managed separately, and coalescing becomes non-trivial when adjacent blocks belong to different size classes.

Trade-offs

- Better balance between concurrency and memory reuse.
- More complex to debug; coalescing logic can become inconsistent.

- Risk of internal fragmentation due to fixed-size bins.

6. Strategy C: Opportunistic Locking and Batching

You can also optimize without restructuring the heap. Instead of locking on every allocation, you can attempt `trylock` first, deferring allocations when the lock is busy or batching multiple allocations under a single lock hold.

```
void *umalloc(size_t size) {
    if (pthread_mutex_trylock(&mLock) != 0) {
        // Lock busy, perform local caching or yield
        sched_yield();
        pthread_mutex_lock(&mLock);
    }
    void *ptr = _umalloc(size);
    pthread_mutex_unlock(&mLock);
    return ptr;
}
```

Alternatively, you could hold the lock for several small allocations at once:

```
void *alloc_many(size_t size, int count) {
    pthread_mutex_lock(&mLock);
    void *ptrs[count];
    for (int i = 0; i < count; i++)
        ptrs[i] = _umalloc(size);
    pthread_mutex_unlock(&mLock);
    return ptrs[0];
}
```

These techniques are heuristic—they trade fairness for throughput and may help on workloads with frequent small allocations.

7. Measuring and Comparing Performance

Once you have implemented an optimization, run both `sharedhash` and `esharedhash` on increasingly large inputs and compare the results.

```
time ./sharedhash test_1MB.bin -t
time ./esharedhash test_1MB.bin -t
```

Then, try scaling the number of threads by increasing the number of blocks (larger files will automatically create more threads). You should see a measurable improvement—perhaps modest (2–3×) for fine-grained locking, or dramatic (5–8×) for per-thread pools. You can instrument your allocator to print lock acquisition counts and correlate them with performance differences.

8. Guidelines for Experimentation

Treat this part as an open-ended experiment. You are not required to reach a specific speedup, but you must reason about the design you chose and demonstrate that you understand the underlying trade-offs. Follow these guidelines:

- Do not optimize at the expense of correctness. The hash output must remain identical.
- Always re-run the single-threaded baseline before claiming a speedup.
- Use small test cases to verify correctness, then large ones to observe performance effects.
- Document any unexpected results—some optimizations can *decrease* performance due to cache contention or allocator overhead.
- Use timing data and observations, not guesswork, in your write-up.

9. Deliverables for Part 2

You must submit the following:

- Your modified **esharedhash.c** source file.
- A short **performance report** (markdown is required) including:
  - Which optimization strategy you implemented and why
  - Timing data comparing `sharedhash` vs. `esharedhash`
  - A short discussion of what you learned about concurrency and performance

10. Grading Criteria

- Correctness: output must match baseline exactly.
- Performance Analysis: shows measurable difference and explains it.
- Design Rationale: describes reasoning for chosen strategy.
- Code Clarity and Documentation: clean, well-commented code.

11. Closing Thoughts

Part 2 represents the transition from building *correct* concurrent programs to designing *efficient* ones. There is no single best solution; each approach involves a balance between simplicity, safety, and scalability. By the time you finish, you should understand not just how to synchronize access to shared memory, but why performance often lags behind theoretical parallelism. This insight—the difference between concurrent and truly parallel execution—is the most valuable lesson in this exercise.

Portfolio Use

This assignment grants you permission to include your completed work in your professional portfolio. You may present your final version, with proper attribution, as an example of your ability to design and implement concurrent systems. Treat this project as a chance to produce something you'll be proud to share, well-structured, functional, and thoughtfully documented.